

U-Ride

Plataforma Institucional de Carpooling Universitario

Matriz Completa de Requisitos Funcionales y No Funcionales del Sistema

Extraída íntegramente del código fuente del proyecto

Universidad Técnica de Ambato

Facultad de Ingeniería en Sistemas, Electrónica e Industrial

Asignatura: Gestión de Pruebas e Implantación de Software

Índice

1. Requisitos Funcionales (RF)	2
2. Requisitos No Funcionales (RNF)	6
3. Resumen de Cobertura	8

Requisitos Funcionales (RF)

Cód.	Nombre	Descripción	Archivo Fuente
RF1	Registro Institucional	El sistema valida que el correo proporcionado termine en @uta.edu.ec. Si no cumple, retorna HTTP 403. Verifica que el correo no esté registrado previamente (HTTP 409 si duplicado). Los campos nombre, email y contraseña son obligatorios.	authController.js:19
RF2	Inicio de Sesión Seguro	El sistema busca al usuario por correo, compara la contraseña ingresada contra el hash almacenado usando bcrypt.compare. Emite un JWT firmado con el id y rol del usuario, con expiración de 24 horas.	authController.js:61
RF3	Gestión de Perfil de Usuario	El usuario puede actualizar: carrera, teléfono, foto de perfil (imagen procesada por multer), coordenadas de zona de referencia (zona_lat, zona_lon) y cambiar de rol entre PASAJERO y CONDUCTOR. La respuesta incluye estadísticas de viajes realizados.	userController.js ProfileScreen.tsx
RF4	Captura de Ubicación GPS	Desde la pantalla de perfil y publicación de viaje, el sistema solicita permiso de ubicación del dispositivo y obtiene las coordenadas actuales con expo-location. Realiza geocodificación inversa para mostrar la dirección en texto legible al usuario.	ProfileScreen.tsx:46 PublishRideScreen.tsx:33
RF5	Publicación de Viaje (Conductor)	El conductor marca origen y destino en un mapa interactivo (MapView) tocando la pantalla. Registra: origenLat, origenLon, destinoLat, destinoLon, fecha_salida, cupos_disponibles (≥ 1), notas_reglas (obligatorio) y costo_contribucion (opcional).	rideController.js:7 PublishRideScreen.tsx
RF6	Validación de Fecha de Publicación	Antes de enviar el viaje al servidor, el frontend verifica que la fecha y hora de salida no sea anterior al momento actual menos 60 segundos. Si el conductor ingresa una fecha pasada, se muestra una alerta y se bloquea la publicación.	PublishRideScreen.tsx:59

Cód.	Nombre	Descripción	Archivo Fuente
RF7	Búsqueda de Viajes por Zona (Pasajero)	El pasajero busca viajes activos enviando su latitud y longitud. El motor SQL usa la fórmula Haversine para calcular distancias y retorna máximo 25 resultados ordenados por proximidad. Solo aparecen viajes con <code>estado = 'ACTIVO'</code> y <code>cupos > 0</code> .	<code>rideController.js:85</code> <code>HomeScreen.tsx:23</code>
RF8	Filtrado de Viajes por Distancia y Fecha	El pasajero puede filtrar los viajes disponibles por radio (1 km, 3 km, 5 km, 10 km) y por fecha opcional mediante un modal de filtros. Los filtros se aplican en la petición al backend. Por defecto el radio es 3 km.	<code>HomeScreen.tsx:195</code>
RF9	Visualización Obligatoria de Reglas del Viaje	Al tocar una tarjeta de viaje, el pasajero ve un bottom-sheet modal con las reglas/notas del conductor antes de poder solicitar el cupo. Solo desde ese modal se puede enviar la solicitud. El campo <code>notas_reglas</code> es NOT NULL a nivel de base de datos.	<code>HomeScreen.tsx:156</code> <code>rideController.js:36</code>
RF10	Solicitud de Cupo (Pasajero)	El pasajero envía una solicitud al conductor. El sistema verifica: que su cuenta no esté suspendida, que el viaje esté activo con cupos disponibles, que no sea el propio conductor y que no haya enviado ya una solicitud al mismo viaje. Se crea en estado <code>PENDIENTE</code> .	<code>requestController.js:6</code>
RF11	Bandeja de Solicitudes del Conductor	El conductor consulta sus solicitudes entrantes en estado <code>PENDIENTE</code> , visualizando nombre y reputación promedio de cada pasajero. Puede aceptar o rechazar cada postulación. Solo el dueño del viaje puede gestionar sus propias solicitudes.	<code>requestController.js:53</code> <code>DriverScreen.tsx</code>
RF12	Confirmación Transaccional de Cupo	Al aceptar una solicitud, el sistema ejecuta una transacción ACID (<code>BEGIN/COMMIT/ROLLBACK</code>) con bloqueo de fila (<code>FOR UPDATE</code>) que: decrementa <code>cupos_disponibles</code> en 1, actualiza el estado de la solicitud a <code>ACEPTADO</code> y registra el evento en auditoría.	<code>requestController.js:66</code>

Cód.	Nombre	Descripción	Archivo Fuente
RF13	Seguimiento de Solicitudes (Pasajero)	El pasajero consulta todas sus solicitudes enviadas con sus estados (PENDIENTE , ACEPTADO , RECHAZADO), la fecha del viaje y el estado del viaje. Si la solicitud fue aceptada y el conductor tiene teléfono registrado, aparece un botón de llamada directa.	requestController.js:15 PassengerRequestsScreen.
RF14	Calificaciones y Motor de Reputación	Tras un viaje, el usuario puede calificar a otro participante con 1 a 5 estrellas y comentario. El sistema bloquea auto-calificaciones y calificaciones duplicadas por viaje. Recalcula automáticamente el <code>reputacion_promedio</code> con <code>AVG</code> redondeado a 2 decimales, dentro de la misma transacción.	reviewController.js
RF15	Reporte de Conducta Indebida	Cualquier usuario autenticado puede denunciar a un participante de un viaje previo. Debe seleccionar el viaje, seleccionar al participante de la lista y describir el motivo con mínimo 20 caracteres. Puede adjuntar evidencia fotográfica. El sistema impide auto-reportes.	reportController.js ReportScreen.tsx
RF16	Historial de Viajes para Reportes	Antes de reportar, el sistema carga el historial completo de viajes del usuario (como conductor o pasajero aceptado) para que el reporte esté contextualizado en un viaje real. Luego carga los participantes del viaje seleccionado.	rideController.js:165 rideController.js:205
RF17	Administración: Revisión de Reportes	El administrador accede al portal web exclusivo y visualiza todos los reportes con sus detalles expandibles: motivo completo, evidencia, denunciante, denunciado, fecha exacta y resolución aplicada. Las filas son expandibles con clic.	adminController.js:8 App.jsx (frontend)
RF18	Administración: Advertencia al Infractor	El administrador aplica una advertencia formal. El reporte se cierra con <code>estado = 'RESUELTO'</code> y <code>resolucion_admin = 'ADVERTENCIA'</code> . El usuario conserva su cuenta activa pero recibe una notificación en su pestaña de Solicitudes con el motivo del reporte.	adminController.js:54

Cód.	Nombre	Descripción	Archivo Fuente
RF19	Administración: Suspensión de Usuario	El administrador suspende una cuenta: la marca como <code>activo = FALSE</code> , cierra el reporte con <code>resolucion_admin = 'SUSPENSION'</code> y registra la acción en auditoría. El usuario suspendido puede iniciar sesión para leer la notificación pero no puede publicar viajes ni solicitar cupos.	<code>adminController.js:27</code>
RF20	Notificaciones de Infracciones al Infractor	Los usuarios ven en su pestaña de Solicitudes una tarjeta especial si han recibido una resolución administrativa. La tarjeta muestra si fue <code>ADVERTENCIA</code> (naranja) o <code>SUSPENSION</code> (roja) con el motivo completo del reporte que la originó.	<code>reportController.js:49</code> <code>DriverScreen.tsx</code> <code>PassengerRequestsScreen</code>
RF21	Cierre Automático de Viajes Expirados	Un proceso en segundo plano se ejecuta cada 60 segundos y cambia automáticamente a <code>estado = 'CERRADO'</code> todos los viajes cuya <code>fecha_salida</code> ya haya transcurrido y que estén en estado <code>ACTIVO</code> . No requiere intervención manual del conductor.	<code>cronJobs.js:10</code>
RF22	Persistencia de Sesión en la App Móvil	El token y los datos del usuario se persisten en <code>AsyncStorage</code> del dispositivo. Al reabrir la app, el sistema restaura automáticamente la sesión sin que el usuario tenga que iniciar sesión de nuevo.	<code>userStore.ts:37</code>
RF23	Procesamiento de Contribución Económica (Simulado)	El sistema acepta un token de tarjeta (<code>token_stripe</code>) y un monto para simular el cobro de la contribución al conductor. Valida el token y retorna un número de recibo simulado (<code>tx_xxxxxxx</code>). En modo prueba, rechaza explícitamente <code>tok_chargeDeclined</code> .	<code>paymentController.js</code>
RF24	Rastreo en Tiempo Real (WebSocket)	El conductor emite su ubicación en tiempo real al servidor mediante el evento <code>update_location</code> . El servidor difunde (<i>broadcast</i>) las coordenadas a todos los pasajeros unidos al canal del viaje (<code>join_ride</code>). Implementado con <code>Socket.io</code> para comunicación bidireccional.	<code>trackerSocket.js</code>

Requisitos No Funcionales (RNF)

Cód.	Nombre	Evidencia en el Código	Archivo Fuente
RNF1	Cifrado de Contraseñas	Las contraseñas se cifran con <code>bcrypt</code> aplicando un salt de 10 rondas antes de insertar en la BD. Nunca se almacena la contraseña en texto plano.	<code>authController.js:34</code>
RNF2	Autenticación con JWT	Todos los endpoints de la API (excepto registro y login) están protegidos por <code>authMiddleware.js</code> , que verifica el token JWT de la cabecera <code>Authorization: Bearer . Tokens</code> inválidos o expirados retornan HTTP 400. El token expira en 24 h.	<code>authMiddleware.js</code>
RNF3	Control de Acceso por Rol Administrativo	Las rutas del panel administrativo (<code>/api/admin/*</code>) están doblemente protegidas: primero por <code>authMiddleware</code> (JWT válido) y luego por <code>adminMiddleware</code> , que rechaza con HTTP 403 si el rol no es <code>ADMINISTRADOR</code> .	<code>adminMiddleware.js</code>
RNF4	Privacidad Geoespacial	El sistema no expone coordenadas GPS exactas de domicilios. El motor de búsqueda trabaja con radio de proximidad (3 km por defecto). La fórmula Haversine se calcula en el servidor, no en el cliente.	<code>rideController.js:124</code>
RNF5	Trazabilidad y Auditoría	Toda acción crítica genera un registro en <code>logs_eventos</code> con: <code>usuario_id</code> , <code>tipo_evento</code> y <code>detalles</code> en <code>JSONB</code> . Eventos registrados: <code>PUBLICACION_VIAJE</code> , <code>ACEPTACION_CUPO</code> , <code>SUSPENSION_ADMINISTRATIVA</code> , <code>EMISION_ALARMA_REPORTE</code> .	<code>rideController.js:67</code> <code>requestController.js:107</code> <code>adminController.js:40</code> <code>reportController.js:32</code>
RNF6	Integridad Transaccional ACID	Las operaciones de aceptación de cupo y calificaciones usan transacciones explícitas con <code>BEGIN/COMMIT/ROLLBACK</code> . El bloqueo <code>FOR UPDATE</code> previene condiciones de carrera en solicitudes simultáneas. En caso de error, el <code>ROLLBACK</code> protege la integridad de los cupos.	<code>requestController.js:55</code> <code>reviewController.js:36</code>

Cód.	Nombre	Evidencia en el Código	Archivo Fuente
RNF7	Restricciones de Integridad en Base de Datos	El esquema SQL impone: <code>UNIQUE(email)</code> , <code>UNIQUE(viaje_id, pasajero_id)</code> , <code>UNIQUE(viaje_id, evaluador_id, evaluado)</code> <code>CHECK(calificacion BETWEEN 1 AND 5)</code> , <code>CHECK(cupos_disponibles >= 0)</code> y <code>NOT NULL</code> en campos críticos.	<code>schema.sql</code>
RNF8	Usabilidad: Teclado y Formularios	Los formularios de la app móvil usan <code>KeyboardAvoidingView</code> con <code>behavior="padding"</code> para evitar que el teclado virtual tape los campos de entrada. Los componentes de formulario (campos reutilizables) se definen fuera del componente principal para evitar re-montajes en cada teclazo.	<code>PublishRideScreen.tsx:82</code> <code>RegisterScreen.tsx</code>
RNF9	Sistema de Diseño Consistente	Todos los componentes visuales de la app móvil usan tokens centralizados: colores, fuentes (6 variantes de <code>Outfit</code>), radios de borde y sombras. La interfaz es compatible con muescas de Android e iOS mediante <code>SafeAreaView</code> .	<code>theme/design.ts</code>
RNF10	Compatibilidad Multiplataforma Móvil	El código de la app detecta <code>Platform.OS</code> para ajustar el <code>paddingTop</code> de los encabezados según Android o iOS, usando <code>StatusBar.currentHeight</code> como referencia en Android. El mapa usa <code>PROVIDER_GOOGLE</code> para consistencia entre plataformas.	<code>HomeScreen.tsx:254</code> <code>DriverScreen.tsx:111</code>
RNF11	Restricción de Dominio Institucional	El registro solo permite correos con extensión <code>@uta.edu.ec</code> . Esta validación se aplica en el servidor y no puede eludirse desde el cliente. La regla institucional está comentada explícitamente en el código.	<code>authController.js:19</code>
RNF12	Escalabilidad Modular del Backend	El backend está organizado en capas separadas: <code>/controllers</code> , <code>/routes</code> , <code>/middlewares</code> , <code>/config</code> y <code>/sockets</code> . Cada módulo funcional tiene su propio controlador y archivo de rutas independiente.	<code>backend/src/</code> (estructura)

Cód.	Nombre	Evidencia en el Código	Archivo Fuente
RNF1	Disponibilidad del Motor de Automatización	El cron job se inicializa en el arranque del servidor (<code>server.js</code>) y es independiente del flujo de peticiones HTTP. Ante un error interno del cron, el servidor registra el fallo en consola pero no se detiene.	<code>cronJobs.js</code> <code>server.js</code>
RNF14	Validación de Motivo en Reportes	El formulario de reporte aplica una restricción de mínimo 20 caracteres en el motivo, tanto en el cliente (botón deshabilitado y contador visible) como en el servidor (validación de campo vacío).	<code>ReportScreen.tsx:83</code> <code>reportController.js:14</code>
RNF15	Comunicación en Tiempo Real (WebSocket)	El servidor inicializa <code>Socket.io</code> junto con el servidor HTTP. Las actualizaciones de ubicación se difunden únicamente dentro del canal de sala (<code>room</code>) del viaje correspondiente, aislando el tráfico entre viajes diferentes.	<code>trackerSocket.js</code> <code>server.js</code>

Resumen de Cobertura

Categoría	Códigos	Cantidad
Requisitos Funcionales	RF1 – RF24	24
Requisitos No Funcionales	RNF1 – RNF15	15
Total		39

Todos los requisitos fueron extraídos directamente del código fuente:

*backend/src/controllers/, backend/src/middlewares/,
backend/src/models/schema.sql,
backend/src/sockets/, backend/src/config/cronJobs.js,
mobile/src/screens/, mobile/src/store/userStore.ts*

Universidad Técnica de Ambato — 2026